Practical Type Inference: High-Throughput Recovery of Real-World Structures and Function Signatures

Lukas Seidel
RevEng.AI & TU Berlin
Germany, Berlin
lukas.seidel@reveng.ai

Sam L. Thomas
RevEng.AI
United Kingdom, Birmingham

Konrad Rieck
TU Berlin & BIFOLD
Germany, Berlin

arXiv:2603.08225v2 [cs.CR] 26 May 2026

Abstract

The recovery of types from stripped binaries is a key to exact decompilation, yet its practical realization suffers. For composite structures in particular, both layout and semantic fidelity are required to enable end-to-end reconstruction. Many existing approaches either synthesize layouts or infer names post-hoc, which weakens downstream usability. This is further aggravated by an excessive runtime overhead that is especially prohibitive in automated environments. We present XTRIDE, an improved n-gram-based approach that focuses on practicality: highly optimized throughput and actionable confidence scores allow for deployment in automated pipelines. When compared to the state of the art in struct recovery, our method achieves comparable performance while being between 70 and 2300× faster. As our inference is grounded in real-world types, we achieve the highest ratio of fully-correct struct layouts. With an optimized training regimen, our model outperforms the current state of the art on the DIRT dataset by 5.09 percentage points, achieving 90.15% type inference accuracy overall. Furthermore, we show that n-gram-based type prediction generalizes to function signature recovery: conducting a case study on embedded firmware, we show that this efficient approach to function similarity can assist in typical reverse engineering tasks.

Keywords

type recovery, binary analysis, artificial intelligence, performance

ACM Reference Format:
Lukas Seidel, Sam L. Thomas, and Konrad Rieck. 2026. Practical Type Inference: High-Throughput Recovery of Real-World Structures and Function Signatures. In *Proceedings of (PREPRINT of Work accepted at ACM CODASPY '26).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Binary reverse engineering is critical for security applications such as malware analysis, vulnerability detection, general code understanding and automated program analysis. A core challenge in this field is the recovery of structural information from binaries, especially stripped ones where variable types, names, and data structures are irrevocably lost during compilation. Recovering this missing type information is essential for producing readable, analyzable decompiler output that can support downstream tasks.

While recent advances in machine learning and static analysis have improved type recovery capabilities, practical deployment remains challenging. Particularly for user-defined structures, both layout fidelity and semantic information are required. Many existing approaches either synthesize layouts without recovering meaningful names [36], or leverage large language models that incur prohibitive runtime overhead [6, 35]. Furthermore, most type recovery systems do not provide a principled mechanism for estimating prediction confidence, making it difficult to filter unreliable predictions or to integrate type recovery into automated pipelines.

Recent work has explored diverse approaches to this problem. Traditional constraint-solving methods [1, 20, 36] rely on costly inter-procedural analysis with substantial computational overhead, often requiring hundreds of seconds per binary. Machine learning approaches using Transformers [6] or Graph Neural Networks [37] can predict types from training corpora but struggle with user-defined structures not seen during training and lack interpretable confidence measures. Hybrid systems combining static analysis with large language models (LLMs) [27, 34, 35] have achieved improved layout recovery and relationship inference at the cost of inflated inference runtimes, focusing on structs as one of the most interesting class of types to recover.

N-gram-based approaches offer a compelling alternative. The STRIDE system [12] demonstrates that matching local token contexts against a database of decompiled code can achieve competitive accuracy while being orders of magnitude faster than LLM-based methods. However, STRIDE provides only a heuristic score without practical thresholding and no differentiated analysis of its performance on real types, as commonly used benchmarks are often biased towards primitives [37]. Consequently, our understanding of the approach in practice is limited by open gaps in training regimen design, type-vocabulary coverage, systematic performance characterization, and calibrated confidence estimation for reliable filtering. These gaps matter most in unsupervised, pipeline-scale settings, where incorrect type application compounds through decompiler propagation and where per-function latency directly determines whether deployment is feasible.

In this paper, we present **XTRIDE**, an improved n-gram-based type recovery system that addresses these limitations. We focus on deployability rather than open-world recovery. XTRIDE targets environments with recurring real types, such as libraries, firmware stacks, and standard software components, where grounded prediction can be integrated end to end into a decompiler's type system.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PREPRINT of Work accepted at ACM CODASPY '26,
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

Our evaluation on DIRT shows that XTRIDE achieves 90.15% general type prediction accuracy, with 98.26% on in-training functions and 68.66% on out-of-training functions, statistically significantly improving over the state of the art by 5.09 percentage points. At the same time, our Rust implementation processes the DIRT test set at 0.04 ms per function, compared to STRIDE's reported 8.2 ms per function and DIRTY's reported 200–8500 ms per function, enabling throughput that is suitable for continuous analysis and large binary corpora. For function-signature recovery on embedded firmware samples, XTRIDE achieves up to 61.27% precision for out-of-train binaries, demonstrating practical utility for identifying key function candidates for initial analysis.

We compare XTRIDE to HyRES [27] and TypeForge [34] on the recovery of struct types. While these systems excel at recovering structure relationships and semantic information, they prioritize comprehensive syntactic recovery over speed. XTRIDE occupies a different point in the design space by trading exhaustive layout recovery on out-of-distribution types for orders-of-magnitude faster inference while maintaining competitive layout recovery performance and providing fully qualified type names from a closed vocabulary. This makes XTRIDE particularly suitable for automated pipelines and large-scale security scanning, where runtime constraints and predictable abstention are first-class concerns.

In summary, we make the following contributions:

- **XTRIDE:** an improved variant of STRIDE with upgraded training regimens, optimized database configurations, and throughput-oriented implementation choices that increase accuracy on DIRT while maintaining inference efficiency.
- **Analysis of Applicability:** We evaluate XTRIDE's performance on recovering complex struct types from real-world binaries. We compare its ability to identify and recover the layout of structs against the state-of-the-art systems HyRES and TypeForge.
- **Definite Confidence Score:** Introducing a calibratable confidence score for threshold-based filtering, we report specific thresholds and their precision-recall trade-offs to enable reproducible filtering strategies in unsupervised deployment.
- **Function Signature Recovery:** We present an experimental extension that applies n-gram matching to function signatures, evaluated on embedded firmware samples as an auxiliary task.

2 Background

In this section, we provide the necessary technical background for understanding type recovery in binary reverse engineering.

2.1 Reverse Engineering

When a program is compiled to produce a binary, substantial information is lost. The compilation process discards variable names, variable types, comments, and high-level abstractions such as user-defined struct. While some information can be preserved through debug symbols (e.g., DWARF), production binaries are typically stripped to reduce size and inhibit understandability. The process of understanding compiled binaries and recovering lost information from them is called reverse engineering.

Decompilers attempt to programmatically recover this lost information through static analysis. Basic type recovery in mainstream decompilers like IDA Pro and Ghidra can distinguish primitive types (e.g., `int`, `long`, `float`) with reasonable accuracy. However, user-defined composite types such as structs remain largely unrecoverable without additional analysis. Compiler optimizations further complicate recovery by disrupting static patterns, eliminating unused variables, and transforming memory access patterns. This loss of type information greatly inhibits readability and understandability of decompiled code as illustrated in Figure 1. A key insight for structure recovery is recognizing field access patterns in assembly code. When a program accesses a field of a structure, the compiled code typically computes an address using a pattern such as:

$$\text{address} = \text{base} + \text{index} \times \text{element_size} + \text{offset}$$

This pattern, denoted $[A + X \cdot Y + Z]$ in prior work [27], captures array indexing within structures. Static analysis systems use these patterns to infer structure layouts by observing consistent offset accesses from a base pointer. However, incomplete dataflow analysis, indirect accesses, and compiler optimizations can obscure these patterns, leading to missed fields or incorrect layout inference. Missing names for functions and argument types further complicates the task of understanding the program's behavior.

```
v5 = *(_DWORD *)(v3 + 124);
v6 = (*(int (__cdecl **)(int, _DWORD))(*(_DWORD *)(v5 + 80)[...];
v4 = v6;
if ( v6 >= 0 )
{
  (*(void (__cdecl **)(int, _DWORD))(*(_DWORD *)(v5 + 80)[...];
  v7 = *(_BYTE *)(v3 + 5);
  *(_BYTE *)(v3 + 5) = 0;
  v4 = sub_284D(0, 0, 5, v7, 0, 0, 0);
  *(_BYTE *)(v3 + 5) = v7;
  *(void (__cdecl **)(int))(dword_8100 + 136)(2000);
```

```
ParentIf = UsbBulkTransfer->ParentIf;
v6 = ParentIf->HubApi->
  ResetPort(ParentIf, UsbBulkTransfer->ParentPort);
v4 = v6;
if ( v6 >= 0 )
{
  ParentIf->HubApi->
    ClearPortChange(ParentIf, ...);
  Address = UsbBulkTransfer->Address;
  UsbBulkTransfer->Address = 0;
  v4 = UsbCtrlRequest(UsbBulkTransfer, EfiUsbNoData, ...);
  UsbBulkTransfer->Address = Address;
  gBS->Stall(2000u);
```

Figure 1: Motivating example for type recovery on decompiled code. Top: Initial output with opaque offsets and generic function pointers. Bottom: The same code segment after correct structures and types have been applied, revealing meaningful field names and function calls.

2.2 Type Recovery

Early approaches to type recovery utilized dynamic analysis to observe memory access patterns at runtime. Systems like Howard [28]

execute binaries to infer types based on offset consistency and system call arguments. However, dynamic methods are difficult to automate in headless environments and struggle to achieve the code coverage necessary for complete binary recovery. Consequently, static analysis has become the dominant paradigm [1, 20]. While newer methods utilize algebraic subtyping to improve scalability [29], deterministic static analysis faces two significant limitations. First, constraint generation is computationally expensive, often prohibiting large-scale analysis. Second, even those methods that are able to reconstruct structural layouts (e.g., distinguishing a pointer from an integer) cannot recover nominal semantic information (e.g., resolving a layout to struct `sockaddr`). Without access to a known type vocabulary, the output remains semantically opaque to the analyst. To address the semantic gap, probabilistic methods have been adopted to approximate the human reverse engineering process. Graph Neural Networks (GNNs) can utilize dataflow graphs constructed from intermediate representations (IR) to learn usage patterns [37]. Most recently, Large Language Models (LLMs) have been applied to treat type recovery as a translation task, predicting variable types and names with high semantic fidelity.

2.3 Language Models

Treating decompiled code as a natural language allows for the application of probabilistic modeling to the type inference problem from another angle. Under this, variable types are inferred not through rigorous constraint solving, but by recognizing semantic and syntactic patterns in the code surrounding a variable's usage. Transformers [31] currently represent the state of the art in this domain. These architectures rely on self-attention mechanisms to weigh the significance of different tokens within a sequence, allowing them to capture long-range dependencies and global context. In the context of type recovery, Transformer-based models predict types by analyzing the entire function body, effectively translating untyped code into typed code. While highly accurate, this process requires dedicated hardware and results in significant computational cost. N-gram models, a precursor to modern deep learning in natural language processing, offer a lightweight alternative. An n-gram is a contiguous sequence of n items from a given text. In type recovery, this approach models a variable's type based on its immediate local context [12]. Fundamentally, both architectures leverage the distributional hypothesis: the assumption that a token's meaning (or a variable's type) can be derived from the context in which it appears. As these approaches model the problem as a classification task, they are limited to the vocabulary of the training data. Hence, they are applicable to inferring types from common libraries but cannot generalize as well as static analysis approaches. However, they come with the benefit of adding not only type layout information but also real-world grounded names.

3 Motivation

We motivate the need for XTRIDE by examining limitations in existing type recovery systems, particularly around confidence calibration, runtime efficiency, and practical deployment.

Runtime efficiency for large-scale analysis. Many modern type recovery systems rely on large language models or complex static analysis. DIRTY requires over one second per function on GPU and 8+ seconds on CPU [6]. For an average UEFI firmware image with 300-500 modules and 50,000+ functions, this translates to multiple days of processing time on CPU and half a day on GPU. Such overhead is prohibitive for large-scale security scanning platforms or continuous integration pipelines. Constraint-solving approaches like OSPREY incur similarly high costs, averaging 528.24 seconds per binary on the Coreutils benchmark [36]. While these methods achieve high accuracy on layout recovery, their computational requirements limit deployment at scale. N-gram-based approaches offer a compelling alternative. STRIDE demonstrates that matching decompiled code tokens against a database can achieve 85% overall accuracy on DIRT with only 8ms per function in Python [12]. While this is a significant improvement over other approaches, it still leaves substantial cost at pipeline scale. A clearer focus on performance is needed to enable new use cases such as security platforms integrating type recovery into every decompilation request or continuous integration pipelines running type inference every commit.

Table 1: Survey of type recovery systems. Overhead refers to runtime cost. Struct Layouts indicates whether the system recovers full struct member information.

System	Approach	Dataset	Overhead	Struct Layouts
IDA [14] & Ghidra [21]	Static Analysis + Heuristics	-	Low	Limited
OSPREY [36]	Constraint	Coreutils	Very High	✓
TypeMiner [18]	Classifier (RF + SVM)	Coreutils	Medium	✗
DIRTY [6]	Transformer	DIRT	High	✓ (in-train)
TYGR [37]	GNN	TYDA	Medium	✗
STRIDE [12]	N-gram	DIRT	Low	✓ (in-train)
HyRES [27]	Static Analysis + LLM	Coreutils	High	✓
ReSym [35]	LLM + Static Analysis	Custom	High	✓
TypeForge [34]	Constraint + LLM	Coreutils	Medium	✓
ByteTR [16]	GNN	TYDA	Medium	✗

Identifying real types. Early type recovery systems [13, 36] only provided primitive type labels such as `int`, `char`, `pointer` and `struct` but stopped to produce predictions for the layout and member types of a struct. This severely limits their practical usefulness as structs are of special interest. Correctly recovering the layouts and semantics of these composite types greatly facilitates the readability of decompiled code [10]. Those systems that attempt to recover struct members are still prone to error as they usually recover each field individually, often failing to recover the layout entirely correctly. Approaches that treat type recovery as

a multi-label classification task [6, 12] avoid this partially, mapping directly to a vocabulary grounded in real-world information, providing full type qualification. While probabilistic approaches can also attempt to recover layouts of never-before-seen types, vocabulary-grounded prediction is inherently unable to generalize to types not seen during training. This can be partially addressed by larger training corpora, e.g, by training on common standard and open source libraries or fine-tuning on context-specific code bases, but will always remain a limitation. The closed-dictionary approach also provides full semantic information, yielding all type and field names upon prediction. Working on fully-named, real data types further increases comprehensibility of decompiled binaries. Many earlier systems working at field-level lack this kind of information completely. Those that provide a separate mechanism for name recovery introduce significant additional overhead for this task [27]. Identifying not only individual variable types but also functions with their parameter types can further enhance decompilation output.

The need for calibrated confidence. Existing type recovery systems produce predictions that indicate relative ranking among candidates rather than a true estimate of correctness. A user seeking to filter unreliable predictions has no principled way to select a threshold. This limitation is particularly acute in automated pipelines and a common oversight in research where the goal is evaluation as opposed to real-world applicability. However, if the type inference system produces many false positives, it becomes counterproductive. A calibrated confidence score gives the user a tool to trade coverage for reliability, producing fewer but more reliable predictions. This is particularly important once recovered types are applied to a decompiler and propagated across functions, where an incorrect struct with wrong offsets can cause compounding downstream errors.

We survey the field of type recovery systems in Table 1 and observe that practically all available systems either do not support the full layout recovery of structs or introduce an overwhelmingly large overhead. STRIDE demonstrated that n-grams can offer an alternative to more elaborate machine learning or constraint solving techniques. We see several limitations that still hinder practical deployment:

[L1] Lack of confidence scoring. STRIDE's scoring mechanism aggregates n-gram matches and diversity factors to produce a heuristic score. This score is useful for ranking candidates but does not correspond to a probability of correctness. Users cannot reliably filter predictions by threshold.

[L2] Primitive type bias. Primitive types such as `int32_t` and `uint64_t` dominate training data, comprising over 30% of all variables. When a struct type and a primitive type have similar scores, STRIDE's tie-breaking by frequency favors the primitive, potentially leading to missed struct predictions. A thorough investigation of the model's performance on non-primitive / -trivial types is currently missing.

[L3] Limited ablations on training configuration. STRIDE evaluates on DIRT but does not report detailed ablations on training set size, n-gram database composition, or memory-accuracy trade-offs.

Table 2: Key differences between STRIDE and XTRIDE.

Feature	STRIDE	XTRIDE
Actionable Confidence	✗	✓
Function signature recovery	✗	✓
Training optimizations	Limited	Extensive
Struct Recovery Evaluation	Limited	✓
Throughput Optimization	✗	✓

[L4] Generalization beyond training data. N-gram approaches match tokens verbatim, limiting generalization to code patterns not seen during training. Understanding how to improve downstream accuracy through better training regimens is critical. At the same time, these systems should be evaluated honestly as closed-vocabulary methods: they are best suited for partially known environments with repeated real types, not for open-world recovery on arbitrarily shifted distributions.

4 Design

In this section, we detail the key architectural choices and workflow of XTRIDE.

4.1 Overview

XTRIDE follows STRIDE's core approach: treating decompiled code as text and matching variable contexts against a database of known patterns. For each variable occurrence in a function, XTRIDE extracts n tokens to the left and n tokens to the right, forming a context window. This context is matched against n-gram databases of varying sizes, e.g., $n \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 30, 60\}$ for the original STRIDE configuration. These blended n-grams were built from a training corpus of decompiled code with groundtruth type annotations. Before hashing, tokens are normalized to reduce sparsity: literals are replaced with placeholders and call contexts include the full extracted argument-signature tokens used at the call site. Throughout this paper, a match denotes hash equality on these normalized token sequences.

XTRIDE extends this foundation in three ways:

(1) **Improved training:** Larger training sets, optimized database configurations, and separate databases for 32-bit and 64-bit binaries to reduce false positives.
(2) **Definite confidence score:** Calibrated confidence estimation using held-out validation data, enabling threshold-based filtering.
(3) **Function signature recovery:** Extension of n-gram matching to function signatures for auxiliary reverse-engineering tasks.

XTRIDE retains STRIDE's core efficiency and n-gram matching approach while adding calibration, training optimizations, and extensibility to auxiliary tasks, all while reducing inference cost even further. These enhancements enable practical deployment in automated pipelines and interactive workflows. Table 2 summarizes key differences between STRIDE and XTRIDE.

4.2 N-Gram Database Construction

XTRIDE constructs n-gram databases from a training corpus of binaries with debug information as groundtruth labels. Each binary in the training set is first decompiled using a standard decompiler (IDA Pro in our case) with debug symbols to obtain groundtruth types and then tokenized into sequences of tokens. For each variable occurrence, context windows of varying sizes n are extracted and the mapping from (n-gram hash, type) pairs along with occurrence counts are recorded. This process is repeated for all training binaries to build a global database. A candidate type is in the top-k set of a query if it appears among the k highest-scoring labels returned for the queried context after local ranking. Unless stated otherwise, we use $k = 3$. A critical design decision is database granularity. STRIDE uses 16 separate databases for different values of n. XTRIDE explores configurations with fewer databases (cf. Section 6.2) to reduce memory overhead while maintaining accuracy. Our evaluation shows that 4 databases ($n \in \{2, 8, 16, 64\}$) achieve competitive accuracy at the same memory requirements when trained on more samples.

Bitness separation. For XTRIDE, we maintain separate databases for 32-bit and 64-bit binaries where possible. Structure access patterns differ between architectures due to pointer size and alignment differences. By separating databases, we reduce false positives where a struct access pattern in 32-bit code matches a different struct in 64-bit code. This design choice is informed by the observation that binary type databases are most effective when fully qualified types resolve to exactly one type per bitness.

4.3 Improved Scoring and Aggregation

XTRIDE inherits STRIDE's scoring mechanism to rank local candidates but introduces a refined confidence for global ranking and interpretation. For each variable occurrence, XTRIDE queries databases for multiple values of n and aggregates matches using three factors: Matches are aggregated by size, where larger n-grams receive higher weights as they capture more specific contexts, frequency, where multiple occurrences of a variable matching the same type increase confidence, and diversity, where n-grams that consistently appear with a single type are more distinctive than those appearing with many types. With w_i being the weight per n-gram size, f_i the frequency of occurrence, and d_i the diversity factor (inverse of the number of types associated with the n-gram), the raw score s for a candidate type t is:

$$s(t) = \sum_{i=1}^{k} w_i \cdot f_i \cdot d_i$$

Furthermore, XTRIDE introduces a confidence score to enable threshold-based filtering of inference results, addressing L1. The goal is to transform the raw aggregated score into a calibrated probability that accurately reflects the likelihood of correctness. We normalize the aggregation scores as follows:

$$c_{\text{norm}} = \begin{cases} \dfrac{s^* - B}{M - B}, & \text{if } M > 0 \wedge M > B \wedge s^* > B, \\ 0, & \text{otherwise,} \end{cases} \qquad B = \tfrac{1}{2}M$$

where s^* is highest raw score among candidates, M is the context-based maximum possible score, i.e., the number of n-gram contexts in which the candidate appears in the database's top-k results, and B is the baseline threshold given our per-context score range $[0.5, 1.0]$. For each prediction in the validation set, we record the raw score and whether the prediction was correct. Using isotonic regression [2], we can fit a non-decreasing piecewise-constant function that minimizes squared error between predicted probabilities and observed frequencies, mapping raw scores to calibrated probabilities. This non-parametric approach captures non-linear relationships between raw scores and correctness.

Threshold selection. A well-calibrated score satisfies the property that among all predictions with confidence c, approximately c fraction are correct. With this property, users can select a threshold τ to achieve desired precision-recall trade-offs. We explore performance at different thresholds (0.90, 0.65, 0.40, none) to illustrate the trade-off space. For production deployment, the threshold can be chosen based on application requirements: high-precision tasks may use $\tau = 0.9$, while exploratory analysis may use $\tau = 0.65$. If the target test distribution is known in advance, full calibration on an in-domain validation set is recommended.

4.4 Function Signature Recovery

XTRIDE extends n-gram matching to function signature recovery as an auxiliary task. The core insight is that function signatures and calling contexts exhibit similar local patterns to variable usage. We treat function calls like variable names and extract the corresponding context tokens. These contexts are matched against a database of function call contexts with full signatures, i.e., including names and types for the function's parameters, as their type labels built from the training corpus. To avoid type confusion, vocabularies for function signatures and usual types are strictly separated. The matching and scoring procedure mirrors variable type recovery in the first stage. As we have global identifiers for functions, e.g., based on their address, we introduce a second aggregation stage: for every function address, we aggregate all predictions across function contexts and weigh the inferred best signature match by number of evaluated contexts per function. This produces a single binary-wide prediction. Figure 2 illustrates the process.

We do not aim to compete with full embedding-based function similarity approaches but present this technique as a fast, low-cost alternative in lower-stake environments. To the best of our knowledge, this is the first technique to attempt function signature recovery solely based on syntactic analysis of the caller context.

4.5 Inference Procedure

At inference time, XTRIDE tokenizes a decompiled function and, for each variable, extracts contextual n-grams from all occurrences to subsequently query the corresponding n-gram databases. The evidence from these queries is aggregated into a raw score per candidate type, and candidates are ranked accordingly. Raw scores are then calibrated into confidence values; the top-ranked type is emitted only if its confidence exceeds a threshold τ, otherwise XTRIDE abstains. When enabled, a lightweight prioritization heuristic biases the final decision toward struct types. The output is the selected type together with its confidence score.

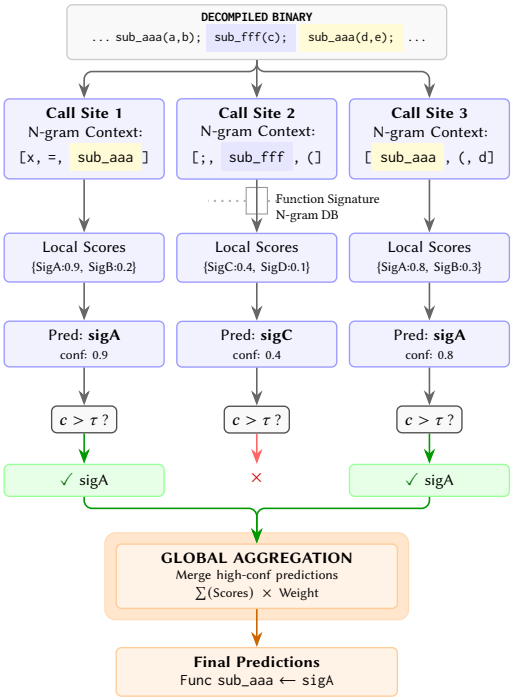


Figure 2: Overview of function signature prediction and global aggregation procedure. Local predictions are filtered by confidence threshold before aggregation.

5 Implementation

XTRIDE is implemented in Rust for efficiency and memory safety. The core n-gram matching engine uses hash maps for $O(1)$ lookup and supports concurrent queries for parallelization. The implementation comprises approximately 5,000 lines of Rust code.

5.1 Feature Extraction and Storage

For each variable and function call occurrence in a decompiled function, we extract the token context consisting of n tokens to the left and n tokens to the right with n varying per database. For calls, we always extract the whole signature including passed arguments, not counting towards the right n. Tokens are normalized to handle variations in whitespace and formatting. Literals (strings, numbers) are replaced with generic placeholders (e.g., `<STRING>`, `<NUM>`) to reduce sparsity.

5.2 Training Configuration

An optimized training procedure is one of the contributions of this work and is explained and evaluated in detail in Section 6.2, addressing L3 and L4. The configuration of the n-gram database is one of the key performance factors. The original STRIDE implementation used 16 databases with sizes {2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 30, 60} without elaborating on this setup. Intuitively, a higher number of smaller context databases allow to match on more local n-grams, improving prediction confidence by having multiple matches. Matching on large contexts, such as 30 or 60 tokens at a time, is rarer but directly leads to high-confidence recognition, basically indicating that an equivalent pattern was seen during training, with low false positive rate. These contexts are stored in separate databases, mapped to their respective type labels. We serialize n-gram databases using `rkyv` [23] to enable fast loading and memory-mapped access. A database ensemble for 64-bit binaries with 300k training samples and 4 n-gram sizes occupies approximately 15 GB on disk and loads in ca. 560ms, where a larger configuration with 16 databases and a total of 48 GB on disk loads in ca. 640ms, averaged across 10 runs.

5.3 Runtime Performance

Our Rust implementation is heavily optimized for performance, profiting from Rust's inherently strong optimizations and speed and making use of multi-processing wherever possible. Memory footprint is minimized at runtime by using memory-mapped I/O for the multi-gigabyte n-gram databases.

6 Experiments

We conducted experiments to answer the following questions:

- **[RQ1]** Can we optimize the training procedure of an n-gram based technique to further improve general type inference performance while maintaining inference speed and memory requirements?
- **[RQ2]** How does the performance of XTRIDE compare with the state of the art in recovering complex struct types?
- **[RQ3]** Can a globally comparable confidence score be helpful in steering the model?
- **[RQ4]** Can XTRIDE's function signature recovery aid in typical reverse engineering tasks?

6.1 Experimental Setup

Datasets. For general type inference, we use the DIRT dataset introduced by the original DIRTY publication [6] as the primary benchmark. It is widely adapted [3, 8, 15, 22] and allows us to directly compare against the current state of the art for vocabulary-based type inference, STRIDE [12]. DIRT comprises binaries compiled from open source C/C++ projects with groundtruth type annotations extracted from debug symbols. The original split consists of 75,656 binaries (738,158 functions) in the train set, and 9,457 binaries (ca. 142,000 functions) in the validation and test set each. For training optimization experiments, we sample additional binaries from the raw unprocessed DIRT dataset, made available by the authors [4]. Additionally, we use a combination of popular real-world binaries to evaluate struct recovery performance: `coreutils`, `wget`, `grep`, `gzip` and `lighttpd`. The binaries used are the exact same versions as in the evaluation of HyRES [26]. We use the IDA-based scripts provided by DIRTY [5] to preprocess the binaries and extract them in a format compatible XTRIDE. All aforementioned datasets consist of binaries compiled for the x86_64 architecture. As all compared systems operate on decompiled code or, in the case of HyRES, on the LLVM intermediate representation [17], we consider the approaches to be architecture-agnostic. For our function recovery case study, we use a dataset of ARM embedded firmware binaries that was used in prior research in firmware emulation [7, 25, 30].

Baseline selection. We compare against **DIRTY** [6], a Transformer baseline that introduced the DIRT dataset, and **STRIDE** [12], the original n-gram-based approach and state of the art on DIRT, for general type inference. For struct recovery, we compare against **HyRES** [27], a hybrid approach specializing on struct recovery, and **TypeForge** [34], an LLM-assisted synthesis-selection method with the same focus, for which we replicate results using the provided open source implementation.

Metrics. For comparative means, we report (1) overall type accuracy as the percentage of variables with correct fully qualified type, (2) struct type accuracy as accuracy on variables with a struct groundtruth, (3) in-training vs. out-of-training performance on functions seen/unseen during training, (4) precision, recall, F1 as per their classical definitions.

Hardware. Unless specified otherwise, experiments were conducted on macOS 15 with an M3 Max CPU with 12 performance cores and 128 GB RAM.

6.2 RQ1: Finding a Pareto-Optimal Training Configuration

For RQ1, we set out to explore if an optimized training procedure can achieve better results within the same constraints. We set the goal as follows: find a better performing combination of number of samples in the training data and number and sizes of produced n-gram databases, such that the final databases are no larger than the databases of the original STRIDE implementation (15 GB) and that the model has no additional overhead in lookup time. We explore various configurations with different sizes, not strictly restricting ourselves to the 15 GB bound, to get a better understanding of the relationship between sample size, n-gram contexts, and performance. Evaluating different training configurations on a dedicated validation set with 20,000 binaries that were not contained in the original train or test sets, Table 3 reports accuracy and memory usage. We also report the throughput in order to give a feeling for performance trade-offs of different database sizes and configurations.

Increasing training set size from 75k to 300k binaries improves overall accuracy from 85.11% to 90.15%. By adjusting the number of databases and their respective token context sizes, we can further minimize memory requirements while covering more unique context samples at the same time. The configuration with 300,000 samples and five databases [48, 12, 8, 4, 2] achieves the best memory to out-of-train accuracy trade-off on the validation set. We designate this configuration XTRIDE, having the same memory requirements as the original STRIDE configuration but better inference speed due to the reduced number of databases. The extended training corpus brings the number of functions that are in training but also in the validation set to ca. 67.58% from STRIDE's 61.05%, but also leads to better context coverage for functions outside the training dataset.

6.3 General Type Inference on DIRT

Table 4 presents overall results on DIRT for XTRIDE and selected baselines. Testing is performed on the original DIRT test, a corpus of 142,193 functions, identical to the one used in previous work [6, 12, 27]. We report overall type accuracy as the percentage of variables with correct fully qualified type as our main metric in accordance with previous work [6, 12], not further differentiating any class bias (**L2**). We replicate experiments for STRIDE (original configuration and databases ran in our Rust implementation) and take metrics for DIRTY as reported by the authors [6]. Although the n-gram based approaches are mostly stable, there are slight differences in inferred labels due to numeric instabilities. We report the average score across 5 runs in Table 4. XTRIDE achieves 90.15% accuracy with the 5-database configuration ($n \in \{2, 4, 8, 12, 48\}$), improving over STRIDE by 5.09 percentage points statistically significant with the Mann-Whitney U test ($p = 0.040$). Our Rust implementation processes the DIRT test set at 0.04 ms per function, where the original Python STRIDE implementation is reported to take 8.2 ms [12] and DIRTY takes between 200 (low beam setting on GPU) and 8500 ms (high beam setting in CPU), resulting in a 200 to $100,000\times$ speed-up. While the large jump in overall accuracy is largely due to more n-grams being indexed during training, we also achieve better out-of-train accuracy with an improved accuracy of 68.66% on functions that are not in the training set as opposed to STRIDE's 65.5%, improving on **L4**. Moreover, XTRIDE achieves 94.88% precision in recovering struct types vs. DIRTY's 68.6%, although their approach explicitly considers and focuses on these with their data layout encoder. A significant share (15.68%) of all unidentified structs had a groundtruth label outside of XTRIDE's vocabulary.

6.4 RQ2: Comparison to the State of the Art in Struct Recovery

As undifferentiated accuracy might paint an incomplete picture (**L2**), we further evaluate our type inference system regarding the recovery of struct types in real-world binaries. These complex, often user-defined types arguably represent the most interesting class of results, as correct recovery adds the most semantic information and readability to decompiler outputs [10].

HyRES [27] and TypeForge [34] are two recently published type recovery approaches with different angles, but both focusing on struct recovery, achieving state-of-the-art results. For HyRES, we present the evaluation metrics as originally reported by the authors [27]. Replicating the results was not possible due to the extreme amounts of required resources, as the original evaluation was run on a machine with 128 cores and 2TB RAM. To ensure comparability, we run this set of experiments on a dataset being comprised of the exact binaries that were used for HyRES [26]. We replicate TypeForge results on this dataset using the provided open-source implementation [33]. During the evaluation, we encountered multiple challenges and inconsistencies. For one, the open source implementation only provides scripts to extract groundtruth and to run inference but not to evaluate these two against each other. Upon request, the authors were not able to provide the original scripts or raw data for verification purposes. Thus, we implemented evaluation to the best of our abilities. We plan to make our implementations publicly available to allow for further verification and comparison. Secondly, the methodology described in their work seems to differ in details from the provided implementation. For example, they defined *composite types* as "structs and pointers to structs, unions and pointers to unions, and arrays" [34]. Yet, their inference implementation only considers variables with observable

Table 3: Hyperparameter tuning of training set size and database composition on validation set. Values in brackets indicate n-gram sizes for the databases, 16 DB denotes the original configuration.

Configuration	Training Size	Memory (GB)	Overall Accuracy %	Out-Train Accuracy %	Throughput ms/func
STRIDE (16 DB)	75,000	15	73.75	59.87	0.13
100k @ [96, 32, 16, 12, 8, 6, 5, 4, 3, 2]	100,000	11	79.26	62.22	0.11
150k @ [64, 32, 16, 8, 2]	150,000	12	83.11	63.90	0.07
150k @ 16 DB	150,000	27	84.03	66.14	0.28
180k @ [64, 32, 16, 8, 5, 3, 2]	180,000	15	84.63	65.09	0.08
250k @ [64, 24, 8, 2]	250,000	14	86.51	65.46	0.05
250k @ [64, 32, 16, 4]	250,000	18	86.43	65.24	0.07
300k @ [64, 16, 8, 2]	300,000	15	87.99	67.23	0.05
300k @ [48, 12, 8, 4, 2]	300,000	14	88.19	67.97	0.05
400k @ [48, 12, 8, 2]	400,000	16	88.96	67.27	0.06

Table 4: Overall type accuracy on DIRT test set.

System	Overall (%)	In-Train (%)	Out-Train (%)
DIRTY	75.8	89.9	56.4
$STRIDE_{repl}$	85.06	**98.53**	65.51
XTRIDE	**90.15**	98.26	**68.66**

struct field access patterns as these are used as inputs to the constraint generation. This rules out arrays as a class of inferrable type. Finally, we identified and fixed a non-diverging path in the layout propagator, leading to non-terminating loops in some cases (e.g., on our versions of `wget2` and `lighttpd`). Although these limitations contribute to the gap in performance between author-reported results and our replication, upon closer inspection, we are confident that our evaluation fairly reflects the technique's capabilities.

For this task, we trained XTRIDE in a second configuration to explore upper bound capabilities: The configuration features a large 16-database configuration and extended the 300k training set by sampling 25% of the functions in the benchmark binaries. We denote this configuration XTRIDE_{PLUS}. We present this configuration explicitly as an upper-bound, in-environment adaptation setting. It may be seen as a threat to validity when comparing to other systems, but prior work also had significant train-test overlap [6, 12], and this setup reflects the intended usage of n-gram-based techniques, which are inherently data-dependent and not designed to generalize robustly across substantially different distributions. In the optimal case, the model is deployed in an at least partially known environment, allowing it to correctly identify the types that are unique to the dataset. Train-Test overlap ranges between 40.9% (`lighttpd` O0) and 71.7% (`coreutils` O3) with a macro average of 61.0%. The total size of the n-gram databases is 48 GB. To mimic the typical symbol-less reverse engineering scenario and to avoid overfitting on semantics, we manually stripped all occurrences of non-generic variable and function names from the training corpus. The original DIRTY dataset includes non-stripped binaries. We report metrics on the following tasks in Table 5:

Struct identification. For this, we report the fraction of all struct and pointer-to-struct types identified as structs. Differences in performance with regard to optimization level are reported in Table 6. In accordance with TypeForge's implementation and also in alignment with HyRES's methodology, we only consider structs and not arrays or unions. Our results on replicating TypeForge show significantly lower recall than reported in the original work (77 - 82%). We attribute much of this gap to evaluation differences, as TypeForge does not attempt inference for many variables with struct groundtruth if they lack observable struct field access patterns. We count these cases as misses. XTRIDE in its base configuration without grounding in the test set has notable difficulties correctly identifying struct types. As XTRIDE's inference result is a real, fully-qualified type extracted from real-world binaries, recognition of the correct type always means we also get full type and field names. TypeForge has no mechanism to recover the semantics of a type and its fields. HyRES attempts to recover type and field names post-hoc with an LLM. They report to correctly recover 67.7% of field names, where a cosine similarity of the token embeddings > 0.85 is considered correct. HyRES excels in identifying structs with a precision and recall of 0.96 and 0.83 respectively, finding even those cases that do not emit obvious struct field access patterns due to its inter-procedural analysis. The fine-tuned XTRIDE_{PLUS} is strong in correctly identifying structs and can surpass our reproduced TypeForge, being on-par with the F1 score of 0.817 reported in the original work [34] as well.

Layout recovery. Both HyRES and TypeForge define a field-level metric. True positives are correct field offsets, false negatives are missed field offsets, and false positives are incorrect offsets. For the layout recovery metric, a result is counted as a true positive only if both the offset and the inferred width per offset are correct. For XTRIDE, we compute P, R and $F1$ by deconstructing the inferred type class label into its field definitions. In addition, we report the fraction of fully recovered structs for XTRIDE and TypeForge, i.e., recovered structs whose layout matches the groundtruth. The necessary data to compute this metric was not available for HyRES. Notably, XTRIDE performs better on this metric than the baselines, presumably due to its closed vocabulary. Producing the correct label during inference automatically yields a fully-correct layout grounded in an existing type from the training corpus. Typical exceptions are different versions of the same struct or minor type confusions after software revisions. TypeForge's macro-average is heavily influenced by a bad performance on the `lighttpd` binary,

Table 5: Struct identification and layout recovery results. Metrics are a macro-average over all benchmark binaries.

System	Struct Identification			Layout Recovery			Full-Match
	P	R	F1	P	R	F1	Accuracy
HyRES	0.958	0.834	0.891	0.892	0.717	0.795	–
Typeforge	0.738	0.456	0.556	0.663	0.507	0.555	0.201
XTRIDE	0.512	0.251	0.329	0.738	0.828	0.768	0.742
$XTRIDE_{PLUS}$	0.833	0.783	0.807	0.928	0.961	0.944	0.943

Table 6: Struct identification performance of $XTRIDE_{PLUS}$ on binaries of varying optimization levels across `coreutils`, `wget`, `grep`, `gzip` and `lighttpd` (macro-average).

Opt-Level	Precision	Recall	F1
O0	0.878	0.757	0.811
O1	0.901	0.815	0.855
O2	0.838	0.811	0.824
O3	0.724	0.774	0.747

Table 7: Runtime performance in seconds, median of ten runs. Numbers are reported per binary.

Program	HyRES	TypeForge	XTRIDE	$XTRIDE_{PLUS}$
wget	719.07	21.38	0.13	0.31
lighttpd	498.91	240.75	0.36	0.89
grep	293.45	63.47	0.19	0.47
gzip	148.04	24.86	0.13	0.33
coreutils	189.32	9.2	0.05	0.08

achieving only 0.30 and 0.41 precision and recall respectively. The system's performance on the other benchmarks is considerably better with up to 0.868 precision on `wget`. Even without fine-tuning, XTRIDE in its base configuration achieves an F1 of 0.77 for layout recovery. The model shows an inverse precision-recall trade-off to HyRES. With the version with partial knowledge of the test distribution, we achieve even better layout fidelity than HyRES with an F1 score of 0.944.

Runtime comparison. We evaluate the inference speed of the different systems on our real-world benchmark binaries. For HyRES, we again use the numbers as they appear in the original publication and acknowledge the difference in evaluation environments. We report runtime of the static dataflow and datalog analysis, excluding runtime of the LLM passes which introduce a further 1.8× on average. For `coreutils`, the reported runtime is a per-binary average, the dataset consists of 117 unique binaries. The TypeForge implementation does not support multi-processing and is thus run sequentially. We repeated all runs three times and report numbers averaged per project and across optimization levels. The analysis time for `lighttpd` diverges notably from the 84 seconds reported in the original work. This implementation had issues with the infinite loops that we fixed, possibly exposing other characteristics that led to slow divergence. For the readability-based refinement via LLM call, we use gpt5-nano instead of gpt4.1-mini. This slows per-binary runtime but had similar practical throughput due to higher API rate limits in a 25-binary subset from `coreutils`. In that configuration, the readability-based LLM constraint refinement takes another 78 (`grep` O0) to 755 (`lighttpd` O3) seconds. Full results are reported in Table 7.

$XTRIDE_{PLUS}$ processes ca. 3000 functions per second on average, i.e., 0.33 ms per function. HyRES and TypeForge do not report per-function timing but per-binary. Search time increases as the number of n-gram contexts stored in our databases grows (16 vs. 4 databases). Even in larger configurations, our system is orders of magnitudes faster than the other approaches, showing a 70× to 2,300× speed-up. Including LLM passes increases this to at least a 180× speed-up against TypeForge. For semantic recovery, HyRES's approach not only leaves a lot of room for LLM-induced hallucinations but also adds a 1.8× overhead on top of their static analysis passes on average. While full evaluation of function signature recovery is out of scope of this work, we note that a closed vocabulary approach provides full semantic information at no additional cost upon correct classification.

6.5 RQ3: Threshold Analysis

For a fixed threshold t, let K_t be the number of kept (non-abstained) predictions and C_t the number of exactly correct kept predictions. The selective exact accuracy at threshold t is defined as

$$\text{sel-acc}_{\text{exact}}(t) = \frac{C_t}{K_t}.$$

The corresponding variable risk is

$$\text{var-risk}_{\text{exact}}(t) = 1 - \text{sel-acc}_{\text{exact}}(t) = 1 - \frac{C_t}{K_t} = \frac{K_t - C_t}{K_t}.$$

That is, $\text{var-risk}_{\text{exact}}(t)$ is the error rate on the non-abstained subset. Accordingly, $\text{struct-risk}_{\text{exact}}(t)$ is the risk only considering variables with a struct type groundtruth. Figure 3 reports performance at different confidence thresholds for the main XTRIDE model configuration on the stripped DIRTY test set.

We show that the procedure behaves intuitively, allowing for fine-tuning of the threshold depending on the task and context. Increasing the threshold can reduce the error rate significantly at the cost of providing type predictions for fewer variables. The other approaches offer no quantitative measure of reliability allowing to discard low-confidence predictions. If the test distribution is known a priori, we recommend full calibration on an in-domain validation set as described in 4.3. If the distribution remains stable, calibration can be reused across deployments with similar data.

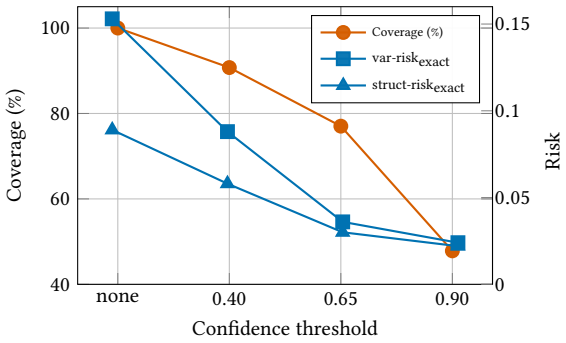


Figure 3: Coverage-Risk trade-offs at different confidence thresholds on the stripped version of the DIRTY benchmark.

Table 8: Function signature recovery results on embedded firmware samples at confidence threshold 0.4.

Sample	Precision	Recall	F1
st_plc	0.604	0.441	0.510
drone	0.567	0.283	0.378
tcp_echo	0.365	0.409	0.386
Average	0.515	0.374	0.431

6.6 RQ4: Function Signature Recovery

For the function signature recovery task as described in 4.4, calls to functions are treated like variables. The surrounding tokens are extracted and matched against n-gram databases, the class label inferred from a separate function vocabulary. On the stripped version of the DIRT test set, XTRIDE$_{PLUS}$ correctly identifies 78.97% of all function calls. This result is heavily biased by the top few percent of frequently occurring functions. Common library functions from popular open-source projects, such as `av_log` from `ffmpeg` with 1.01%, are the most frequently correctly inferred functions.

As performance appears numerically low, we conduct a case study to get a better feeling for its impact in real-world applications: we mimic a typical reverse engineering workflow where we need to identify key functions in the complete absence of symbols, in this case on a set of embedded firmware binaries (cf. 6.1). Hardware Abstraction Layer (HAL) functions often interact with peripherals via MMIO and are of special interest during emulation, rehosting and fuzzing of embedded firmware as they require special handling. They are commonly hooked in rehosting systems like HALucinator [7]. To enable the model to recognize the desired HAL functions, we add two binaries to the training corpus: `p2im_plc`, another common benchmark binary for embedded rehosting [7, 11, 25], and a test project compiled to include as many HAL functions as possible. The first provides real-world usage contexts while the latter maximizes vocabulary coverage of HAL functions. Both these binaries, as well as our benchmark binaries, use common functions from the STM32 HAL library. We do not add any code from the binaries we evaluate on to the training corpus, thus the percentage of test-train overlap lies in the 1.5 - 4.5% range only. Table 8 reports precision, recall, and F1 for identifying HAL functions at threshold 0.4.

Function signature recovery achieves moderate precision but low recall, indicating that the approach is conservative and that many usage patterns do not match contexts in the training corpus. Our function recovery mechanism aggregates labels and produces one final inference result per global function start address, so repeated calls inside a binary are not double-counted at the reporting stage. For groundtruth selection, we consider all functions with names starting with `HAL_`. The `st_plc` binary contains 1127 inferrable functions, 286 of which are HAL functions. Our approach correctly identifies 126 HAL functions with a precision of 60.39%, while 21 HAL functions were missing from the vocabulary, accounting for 13.04% of false negatives. On `tcp_echo`, only 69 of 732 functions are HAL functions, resulting in lower precision. Overall, we correctly identify 44.06, 28.33, and 40.91% of all HAL functions in the binaries for `st_plc`, `drone` and `tcp_echo` respectively.

We consider this result promising, demonstrating utility for identifying likely HAL functions. Especially in self-contained binary blobs, like many embedded firmware are, that contain hundreds of functions, this extremely low-overhead approach is a fast way to condense the number of functions to manually look at when reverse engineering.

7 Discussion

Recovering types for stripped binaries is only useful when the recovered information survives contact with downstream tooling. The results indicate that struct recovery is constrained as much by semantic fidelity and integration requirements as by raw layout reconstruction. Grounding predictions in a closed vocabulary of fully qualified, real-world types yields outputs that are immediately actionable. A correct prediction implies not only a compatible layout but also a stable semantic identifier that can be propagated through decompiler types, signatures, and dataflow views without an additional naming step.

Our results also indicate that improvements are not primarily tied to more elaborate inference, but to how sparsity is managed at training and indexing time. Scaling the corpus while reducing the number of n-gram databases improves test accuracy without expanding the memory footprint, suggesting that database composition is a more effective lever than using many overlapping context indices. This behavior is consistent with the intuition that diverse, well-chosen context lengths improve coverage of predictive neighborhoods in token space, whereas additional databases can contribute redundant evidence with diminishing returns.

For composite structs, fully correct layouts follow at a high rate if a struct is correctly identified, reflecting the advantage of predicting fully-qualified grounded types. The dominant failures arise earlier, at identification, particularly for types absent from the vocabulary. This contrasts with layout synthesis approaches such as HyRES and TypeForge, which infer shapes from program evidence. Grounded prediction offers high semantic fidelity when the ecosystem contains recurring types, while synthesis remains useful when types are novel or definitions are unavailable.

The confidence analysis further bridges benchmark accuracy and deployability. The precision–recall trade-offs obtained via thresholding indicate that the model supports a selective mode in which predictions can be applied automatically at high confidence while

abstaining on ambiguous cases. This is particularly relevant for automated pipelines, where even a small rate of incorrect type application can cascade into misleading decompiler output.

Finally, the embedded firmware case study extends the same usage-based principle to function signature recovery. The observed profile, high precision with modest recall, is consistent with a conservative similarity signal that is useful for triage. Surfacing a small set of high-interest, in this case library or HAL functions can help to anchor manual analysis. In throughput-bound settings, a fast abstaining predictor can prioritize analyst attention without broad noisy labeling.

The main limitation of the n-gram-based approach follows directly from the design choice that enables semantic fidelity, as predictions cannot exceed the closed vocabulary. Misses on out-of-vocabulary types are therefore expected, and improvements in corpus breadth mainly reduce this boundary rather than remove it. Accordingly, we do not claim open-world recovery. The approach is best suited for partially known environments with recurring real types, such as library-rich software stacks, firmware ecosystems, or supply-chain settings with substantial component reuse. The results suggest a natural division of labor where grounded prediction covers the frequent and recurrent portion of real-world type ecosystems with high fidelity and very low overhead and residual cases can be deferred to heavier reconstruction.

8 Related Work

In this section, we discuss related work in type recovery.

Traditional static analysis approaches. Early approaches to type recovery relied on constraint solving and heuristics. Decompilers such as IDA Pro and Ghidra can recover primitive data types with reasonable accuracy [14, 21]. However, these tools struggle significantly with user-defined composite data types, often misidentifying structures as generic integer types.

Constraint-solving methods such as TIE [1], Retypd [20], and OSPREY [36] employ whole-program memory dependency analysis and heuristic rules to infer types and layouts without debug information. OSPREY uses probabilistic inference based on memory access patterns and dynamic dataflow, achieving 90% precision on primitive types on a `coreutils` benchmark. For struct layout recovery, OSPREY achieves F1 0.869 on the `coreutils` dataset, though with substantial computational overhead (averaging 528.24 seconds per binary). Static inference methods in general tend to be computationally expensive, limiting their scalability and practical speed. The BinSub [29] authors acknowledge that existing implementations of type inference systems that are based on subtyping and polymorphism are too inefficient to achieve adoption in production. Their work introduces algebraic subtyping to the domain, leading to a faster method for binary type reconstruction, framing the work as an alternative approach to the static methods in mainstream decompilers rather than as a post-hoc enhancement.

Machine learning approaches. With the rise of machine learning, data-driven approaches have emerged to augment decompilation. Especially Transformers and LLMs have been successful recently in type recovery tasks across domains [6, 9, 24, 32, 35]. DIRTY [6] is a Transformer-based model that post-processes decompiled files to generate variable names and types. It supports 48,888 types from open source C code and uses a Data Layout Encoder to incorporate memory layout information. DIRTY achieves 75.8% overall type accuracy but performance notably decreases on testing sets due to its closed vocabulary approach. Furthermore, DIRTY requires over one second per function on GPU and more than 8× that on CPU, making it prohibitive for large-scale analysis. TYGR [37] uses Graph Neural Networks with intra-procedural dataflow analysis on VEX IR [19] to infer basic and struct variable types from a closed dictionary. TYGR introduces the TYDA dataset due to the high occurrence of undesired artifacts in prior datasets [37]. On the `coreutils` O0 benchmark, TYGR reports overall accuracy of 74.5% and struct accuracy of 40.6%. While TYGR aims to predict struct shapes and member types, other work reports that it fails to recover member layout comprehensively, with low F1 scores (9.1%) for layout recovery in comparison evaluations [34]. ByteTR [16] conducts inter-procedural analysis using its BytePA algorithm and uses a Gated GNN for variable-level semantic representation. It explicitly does not support recovery of full structure layouts or specific structure identifier names, considering this a difficult problem for future work.

Hybrid and advanced approaches. Recent work combines static analysis with LLMs to improve semantic recovery. HyRES [27] is a hybrid reasoning technique combining Datalog-based reasoning on LLVM IR [17] with LLMs and heuristics. The approach performs intra- and inter-procedure dataflow analysis with structure aggregation to overcome incomplete dataflow. It significantly outperforms OSPREY and ReSym [35] in struct pointer identification and layout recovery with substantially lower overhead than OSPREY. ReSym combines fine-tuned LLMs with a lightweight Prolog-based reasoning system. It uses Andersen's algorithm for dataflow analysis on decompiled code and aggregates LLM queries to suppress uncertainty. ReSym substantially outperforms OSPREY and DIRTY in variable name and type recovery. At the same time, its dataflow analysis is inherently undecidable, leading to incomplete type grouping, and identifying type-agnostic arguments is challenging, reducing precision. TypeForge [34] employs a two-stage synthesis-selection strategy. The first stage uses a Type Flow Graph for efficient hint collection with conflict-aware type propagation to prevent erroneous propagation. The second stage uses LLM-assisted readability-guided selection to choose the best-fit type declaration. TypeForge significantly outperforms DIRTY, OSPREY, ReSym, and TYGR in composite data type identification and layout recovery.

9 Conclusion

In this work, we position n-gram-based type recovery as a practical component for high-throughput program analysis workflows. Our system XTRIDE combines an optimized n-gram database portfolio that improves accuracy without inflating memory, thresholdable confidence, and an experimental extension to function signature recovery.

XTRIDE achieves 90.15% overall accuracy and 68.66% out-of-training accuracy, improving over the state of the art by 5.09 and 3.15 percentage points respectively while reducing the database count and keeping the memory footprint consistent. Moreover, our implementation yields 10^3–$10^5\times$ speedups over static-analysis-heavy, LLM-centered pipelines. By thresholding on confidence,

users can trade coverage for reliability and avoid applying low-quality types that would otherwise degrade downstream decompiler output. This places high-fidelity, vocabulary-grounded type annotation within reach of interactive decompilation and large-scale automated analysis. The function recovery case study suggests that the same n-gram matching principle provides a low-cost similarity signal for auxiliary tasks, presenting the first unification of type and function inference into a single approach to the best of our knowledge. We achieve moderate precision (51.47% on average), which supports conservative triage during early binary analysis. Finally, the struct type recovery results demonstrate that grounded prediction yields strong end-to-end correctness if types exist in the vocabulary.

Overall, the results show that n-gram type recovery can offer an effective efficiency-fidelity trade-off in partially known environments. With appropriate training scale, database design, calibrated confidence, and in-domain deployment, it becomes a practical component of robust high-throughput program analysis workflows.

Availability

The source code for the XTRIDE Rust implementation, as well as the trained databases used for our experiments, will be accessible here: https://github.com/pr0me/XTRIDE.

References

[1] Thanassis Avgerinos. 2011. TIE: Principled reverse engineering of types in binary programs. In Symposium on Network and Distributed System Security (NDSS).

[2] Richard E Barlow and Hugh D Brunk. 1972. The isotonic regression problem and its dual. J. Amer. Statist. Assoc. 67, 337 (1972), 140–147.

[3] Kevin Cao and Kevin Leach. 2023. Revisiting deep learning for variable type recovery. In 2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC). IEEE, 275–279.

[4] Qibin Chen, Jeremy Hunt, Kavinda Ravishan Fernando, Xiangzhe Gao, Gene Tsudik, and Ahmad-Reza Sadeghi. 2022. DIRT: Unprocessed full Dataset. https://doi.org/10.1184/R1/20732656 Last Accessed: 10.12.2025.

[5] Qibin Chen, Jeremy Hunt, Kavinda Ravishan Fernando, Xiangzhe Gao, Gene Tsudik, and Ahmad-Reza Sadeghi. 2022. DIRTY: Open Source Version. https://github.com/CMUSTRUDEL/DIRTY/tree/main Last Accessed: 15.12.2025.

[6] Qibin Chen, Jeremy Lacomis, Edward J Schwartz, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. 2022. Augmenting decompiler output with learned variable names and types. In USENIX Security Symposium. 4327–4343.

[7] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. 2020. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. In USENIX Security Symposium.

[8] Luke Dramko, Deniz Bölöni-Turgut, Claire Le Goues, and Edward Schwartz. 2025. Quantifying and Mitigating the Impact of Obfuscations on Machine-Learning-Based Decompilation Improvement. In Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA). Springer, 244–266.

[9] Luke Dramko, Claire Le Goues, and Edward J Schwartz. 2025. Idioms: Neural Decompilation With Joint Code and Type Definition Prediction. arXiv preprint arXiv:2502.04536 (2025).

[10] Haeun Eom, Dohee Kim, Sori Lim, Hyungjoon Koo, and Sungjae Hwang. 2024. R2I: A Relative Readability Metric for Decompiled Code. Proc. ACM Softw. Eng. 1, FSE, Article 18 (July 2024), 23 pages. doi:10.1145/3643744

[11] Bo Feng, Alejandro Mera, and Long Lu. 2020. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In USENIX Security Symposium.

[12] Harrison Green, Edward J Schwartz, Claire Le Goues, and Bogdan Vasilescu. 2024. STRIDE: Simple Type Recognition In Decompiled Executables. arXiv preprint arXiv:2407.02733 (2024).

[13] Jingxuan He, Pesho Ivanov, Petar Tsankov, Veselin Raychev, and Martin Vechev. 2018. Debin: Predicting debug information in stripped binaries. In ACM Conference on Computer and Communications Security (CCS). 1667–1680.

[14] Hex-Rays. 1999. IDA Pro. https://hex-rays.com/ida-pro/. Last Accessed: 15.12.2025.

[15] Peiwei Hu, Ruigang Liang, and Kai Chen. 2024. DeGPT: Optimizing decompiler output with llm. In Symposium on Network and Distributed System Security (NDSS), Vol. 267622140.

[16] Gangyang Li, Xiuwei Shang, Shaoyin Cheng, Junqi Zhang, Li Hu, Xu Zhu, Weiming Zhang, and Nenghai Yu. 2025. Beyond the Edge of Function: Unraveling the Patterns of Type Recovery in Binary Code. arXiv preprint arXiv:2503.07243 (2025).

[17] LLVM 2017. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html

[18] Alwin Maier, Hugo Gascon, Christian Wressnegger, and Konrad Rieck. 2019. Typeminer: Recovering types in binary programs using machine learning. In Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA). Springer, 288–308.

[19] Nicholas Nethercote and Julian Seward. 2007. Valgrind: a framework for heavy-weight dynamic binary instrumentation. ACM Sigplan notices 42, 6 (2007), 89–100.

[20] Matt Noonan, Alexey Loginov, and David Cok. 2016. Polymorphic type inference for machine code. In ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). 27–41.

[21] NSA. 2019. Ghidra: Reverse Engineering Suite. https://ghidra-sre.org/. Last Accessed: 7.12.2025.

[22] Kuntal Kumar Pal, Ati Priya Bajaj, Pratyay Banerjee, Audrey Dutcher, Mutsumi Nakamura, Zion Leonahenahe Basque, Himanshu Gupta, Saurabh Arjun Sawant, Ujjwala Anantheswaran, Yan Shoshitaishvili, et al. 2024. " Len or index or count, anything but v1": Predicting Variable Names in Decompilation Output with Transfer Learning. In IEEE Symposium on Security and Privacy (S&P). IEEE, 4069–4087.

[23] rkyv 2025. rkyv: zero-copy deserialization framework for Rust. https://rkyv.org/

[24] Lukas Seidel, Sedick David Baker Effendi, Xavier Pinho, Konrad Rieck, Brink Van der Merwe, and Fabian Yamaguchi. 2023. Learning Type Inference for Enhanced Dataflow Analysis. In European Symposium on Research in Computer Security (ESORICS). Springer, 184–203.

[25] Lukas Seidel, Dominik Maier, and Marius Muench. 2023. Forming Faster Firmware Fuzzers. In USENIX Security Symposium. https://www.usenix.org/conference/usenixsecurity23/presentation/seidel

[26] Zihan Sha, Hui Shu, Hao Wang, Zeyu Gao, Yang Lan, and Chao Zhang. 2025. HyRES Dataset. https://github.com/Sandspeare/HyRES/tree/main/binaries. Last Accessed: 10.12.2025.

[27] Zihan Sha, Hui Shu, Hao Wang, Zeyu Gao, Yang Lan, and Chao Zhang. 2025. HyRES: Recovering Data Structures in Binaries via Semantic Enhanced Hybrid Reasoning. ACM Trans. Softw. Eng. Methodol. (May 2025). doi:10.1145/3736719

[28] Asia Slowinska, Traian Stancescu, and Herbert Bos. 2011. Howard: A Dynamic Excavator for Reverse Engineering Data Structures.. In Symposium on Network and Distributed System Security (NDSS).

[29] Ian Smith. 2024. Binsub: The simple essence of polymorphic type inference for machine code. In International Static Analysis Symposium (SAS). Springer, 425–450.

[30] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, Christopher Kruegel, Thorsten Holz, and Ali Abbasi. 2022. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In USENIX Security Symposium.

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems 30 (2017).

[32] Ashwin Prasad Shivarpatna Venkatesh, Rose Sunil, Samkutty Sabu, Amir M Mir, Sofia Reis, and Eric Bodden. 2025. An empirical study of large language models for type and call graph analysis in Python and JavaScript. Empirical Software Engineering 30, 6 (2025), 167.

[33] Yanzhong Wang, Ruigang Liang, Yilin Li, Peiwei Hu, Kai Chen, and Bolun Zhang. 2025. TypeForge Open Source Implementation (Ghidra Extension). https://github.com/noobone123/TypeForge. Last Accessed: 10.12.2025.

[34] Yanzhong Wang, Ruigang Liang, Yilin Li, Peiwei Hu, Kai Chen, and Bolun Zhang. 2025. TypeForge: Synthesizing and Selecting Best-Fit Composite Data Types for Stripped Binaries. In IEEE Symposium on Security and Privacy (S&P). IEEE Computer Society, 2847–2864. doi:10.1109/SP61157.2025.00193

[35] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. 2024. Resym: Harnessing llms to recover variable and data structure symbols from stripped binaries. In ACM Conference on Computer and Communications Security (CCS). 4554–4568.

[36] Zhuo Zhang, Yapeng Ye, Wei You, Guanhong Tao, Wen-chuan Lee, Yonghwi Kwon, Yousra Aafer, and Xiangyu Zhang. 2021. Osprey: Recovery of variable and data structure via probabilistic analysis for stripped binary. In IEEE Symposium on Security and Privacy (S&P). IEEE, 813–832.

[37] Chang Zhu, Ziyang Li, Anton Xue, Ati Priya Bajaj, Wil Gibbs, Yibo Liu, Rajeev Alur, Tiffany Bao, Hanjun Dai, Adam Doupé, et al. 2024. {TYGR}: Type Inference on Stripped Binaries using Graph Neural Networks. In USENIX Security Symposium. 4283–4300.

Received 02 January 2026